

Relatório - Web Crawler

Jorge Augusto de Lima e Silva
jorge.lima2407@gmail.com

Universidade Federal de Minas Gerais
Belo Horizonte, Minas Gerais, Brazil

1 Introdução

Web crawlers são ferramentas fundamentais para coleta de dados na web. Este relatório apresenta um crawler multithread em Python, desenvolvido para a disciplina do DCC-UFMG (2025), que coleta páginas respeitando robots.txt e políticas de politeness. A implementação utiliza threads para melhorar desempenho, com foco em: (1) gerenciamento concorrente, (2) verificação automática de permissões, e (3) armazenamento em WARC. Diferente de frameworks como Scrapy, a solução emprega bibliotecas básicas de Python, ilustrando conceitos de concorrência e acesso web. O sistema processa 100.000 páginas com delays dinâmicos e arquitetura modular que separa coleta, processamento e armazenamento.

2 Arquitetura do Sistema

A implementação consiste em quatro componentes principais que trabalham em conjunto para realizar o crawling de forma eficiente e segura. A classe Crawler atua como o núcleo do sistema, coordenando todo o processo de descoberta e coleta de páginas web. Ela mantém uma fila de URLs a serem visitadas (`_frontier`) e utiliza estruturas thread-safe para garantir a consistência dos dados durante o acesso concorrente.

Para gerenciar as políticas de acesso a cada domínio, o sistema emprega o padrão Singleton através da classe Politeness, que centraliza as informações sobre regras de robots.txt e intervalos entre requisições. Cada domínio possui seu próprio objeto PageInfo, que armazena detalhes como o tempo da última requisição e as permissões definidas no arquivo robots.txt correspondente.

A persistência dos dados coletados é tratada pela classe Writer, também implementada como Singleton, que realiza a escrita em formato WARC de forma otimizada, utilizando um sistema de buffering para reduzir a frequência de operações de I/O no disco. Quando o buffer atinge um tamanho pré-definido (100 páginas) ou quando o limite de coleta é atingido, o conteúdo é gravado em arquivos compactados.

3 Mecanismos de Controle de Concorrência

O sistema enfrenta diversos desafios típicos de ambientes multithread, como condições de corrida no acesso às estruturas compartilhadas. Para garantir a consistência, foram implementados vários mecanismos de sincronização. O acesso à fila de URLs (`_frontier`) e aos conjuntos de URLs visitadas/processadas é protegido por um `threading.Lock`, garantindo que apenas uma thread por vez possa modificar essas estruturas.

No gerenciamento de politeness, cada objeto PageInfo possui seu próprio lock, permitindo que diferentes threads acessem domínios distintos simultaneamente, mas serializando o acesso a um mesmo domínio. Isso é particularmente importante durante a fase de inicialização, quando o arquivo robots.txt está sendo baixado e processado.

O sistema de escrita emprega um `threading.RLock` (reentrant lock) para proteger o buffer compartilhado, permitindo que a mesma thread possa adquirir o lock múltiplas vezes quando necessário. A estratégia de buffering reduz a contenção pelo lock, já que as threads normalmente apenas adicionam itens ao buffer, enquanto a operação mais custosa de escrita em disco ocorre de forma menos frequente.

4 Fluxo de Processamento

O ciclo principal de crawling inicia quando o método `crawl()` é invocado, criando várias threads que executam em paralelo a rotina `_crawl()`. Cada thread obtém URLs da frontier de forma aleatória (para melhor balanceamento de carga), verifica as regras de acesso através do módulo de politeness, e realiza o download do conteúdo quando autorizado.

Após o download, o conteúdo é parseado usando BeautifulSoup, que extrai tanto os dados textuais (para armazenamento) quanto os links presentes na página. Os novos URLs descobertos passam por um processo de normalização e validação antes de serem adicionados à frontier, garantindo que apenas URLs válidas e não visitadas sejam incluídas no processo de crawling.

O sistema implementa um mecanismo de espera inteligente baseado no tempo da última requisição para cada domínio, calculado através do método `time_to_wait()` da classe PageInfo. Isso garante o cumprimento dos intervalos mínimos especificados nos arquivos robots.txt, evitando que o crawler seja bloqueado por fazer requisições muito frequentes.

5 Considerações sobre Desempenho

A análise de complexidade revela que o tempo de execução é dominado principalmente pelo processo de download e parseamento de páginas ($O(N \cdot (M + L))$), onde N é o número de páginas, M o tamanho médio do HTML e L o número médio de links por página. A estratégia de seleção aleatória de URLs da frontier ajuda a distribuir uniformemente o acesso entre diferentes domínios, reduzindo gargalos causados por politeness.

O uso de estruturas de dados eficientes (sets para conjuntos de URLs e dicionários para mapeamento de domínios) garante operações $O(1)$ para as verificações mais frequentes no sistema. O buffer de escrita permite amortizar o custo das operações de I/O, enquanto o sistema de locks finamente granularizados minimiza as contenções entre threads.

Em situações onde o arquivo robots.txt não está disponível ou é inválido, o sistema adota valores padrão conservativos (delay de 0.1 segundos), garantindo que o crawling continue de forma responsável mesmo sem informações explícitas de politeness. O tratamento robusto de erros em todas as etapas do processo assegura que falhas pontuais não comprometam a execução como um todo.

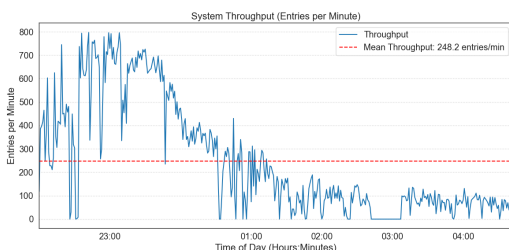


Figure 1: Taxa de Download e Throughput do Sistema

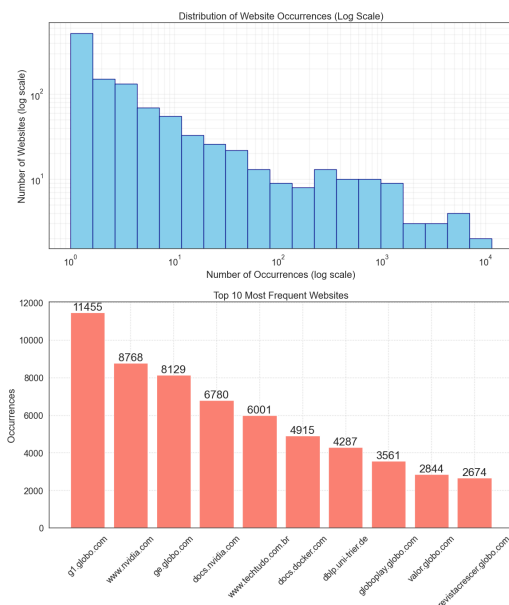


Figure 2: Distribuição de Domínios (Top 10 em destaque)

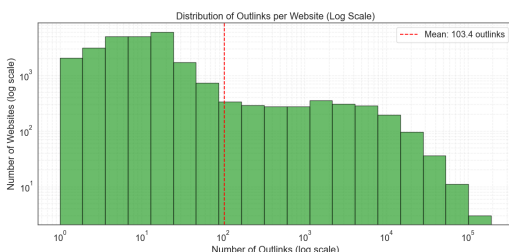


Figure 3: Distribuição de Outlinks por Página (Escala Logarítmica)

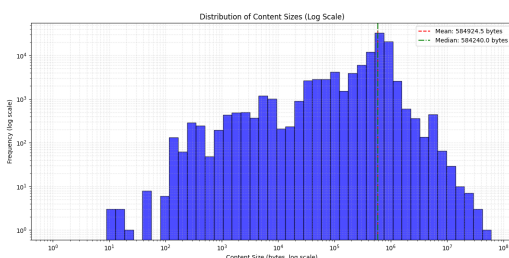


Figure 4: Distribuição do Tamanho de Conteúdo

6 Análise Experimental

A análise da taxa de download (Figura 1) revela um gargalo no programa, evidenciado pelo decaimento gradual ao longo do tempo. No entanto, destaca-se a robustez do sistema, que demonstra resiliência mesmo em situações críticas, como quedas abruptas para 0 (possivelmente devido a erros transitórios ou instabilidades na rede). A média de throughput de **248.2 entradas por minuto** (gráfico "System Throughput") indica um desempenho estável, mas a variação ao longo do período analisado (entre 23:00 e 04:00) sugere flutuações que podem estar relacionadas a carga do servidor ou limitações de hardware.

A distribuição de domínios (Figura 2) segue uma **lei de potência típica de dados na web**, com uma cauda longa onde poucos domínios dominam o corpus. Os 10 domínios mais frequentes (como *www.nvidia.com*, *g1.globo.com* e *docs.nvidia.com*) concentram a maioria das ocorrências, com picos acima de **11.455 entradas** para o domínio mais acessado. Isso reflete a ausência de políticas para limitar a profundidade de exploração ou balancear a coleta entre domínios, resultando em uma super-representação de sites específicos.

A análise de outlinks (Figura 3) confirma a natureza **altamente conectada** das páginas web, com uma média de **103.4 links por página**. A distribuição em escala logarítmica mostra uma dispersão ampla: enquanto a maioria das páginas tem entre 10^1 e 10^3 links, algumas chegam a 10^5 **ou mais**, indicando "hubs" (páginas com extremo número de links). Essa característica é típica de redes scale-free e reforça a necessidade de algoritmos eficientes para navegação e indexação.

O tamanho do conteúdo (Figura 4) apresenta uma distribuição assimétrica em escala logarítmica, com média de **584.924,5 bytes** (585 KB) e mediana de **584.240 bytes** (584 KB), indicando que a maioria dos arquivos se concentra nessa faixa de tamanho. A distribuição mostra que, embora existam alguns outliers maiores, a grande maioria do conteúdo coletado possui tamanhos consistentes próximos à média, o que sugere um padrão dominante de páginas com tamanho moderado. Essa característica é favorável para estratégias de armazenamento e indexação, pois reduz a necessidade de tratamento especial para arquivos extremamente grandes ou pequenos. A proximidade entre média e mediana também indica uma relativa simetria na distribuição central dos dados.

Os dados coletados evidenciam padrões esperados em crawlers web, como a concentração de domínios e a distribuição desigual de outlinks. Contudo, os gargalos na taxa de download e as flutuações de throughput indicam oportunidades para melhorias no gerenciamento de threads, políticas de retry para falhas e balanceamento de carga. Além disso, a análise de tamanho de conteúdo reforça a importância de compressão e armazenamento hierárquico para lidar com a diversidade de dados.

7 Corpus

O corpus pode ser encontrado no link: [aqui](#).